Faculty of Computing

BSc (Hons) in Software Engineering
Year 2 Semester 2 (2026)

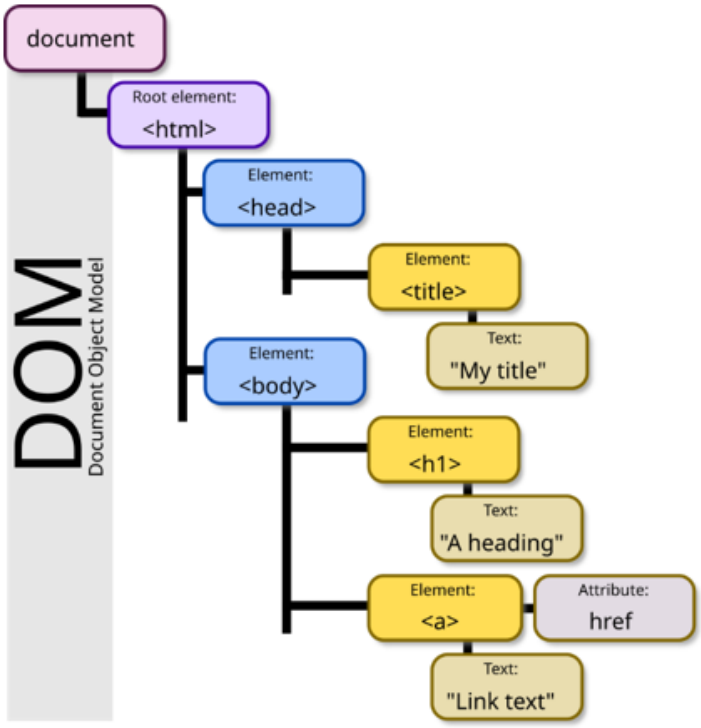
SE2020 - Web and Mobile Technologies Lab 04

Practical Lab: HTML DOM with jQuery

DOM (Document Object Model)

The **DOM (Document Object Model)** is a programming interface for web documents. It represents the structure of HTML and XML documents as a tree of objects that can be manipulated with programming languages like JavaScript.

- The DOM represents your HTML document as a tree structure
- Each HTML element becomes a node in the tree
- JavaScript can access and modify these nodes
- Changes to the DOM update what you see in the browser



1. Scenario: Interactive Travel Registration Portal

1.1 Background

A travel agency named **Travel Adventures** is developing a simple web portal to promote travel services and collect traveler details.

The web page already contains:

- A title and descriptive content about travel
- A travel-related image
- Travel tips and a cost overview table
- A form to collect traveler information (Name, Age, Email)

At the moment, this page is **static**, meaning:

- The content does not change after the page loads
- User interactions do not affect the page behavior
- Submitted form data is not processed or displayed dynamically

Your task in this lab is to **convert this static web page into an interactive web page** using **HTML DOM and JavaScript**.

1.2 What you Achieve by Completing This Lab

By successfully completing this lab exercise, you will be able to:

- Understand the **DOM tree structure** and how the browser represents HTML
- Use **DOM methods and document object properties** effectively
- Dynamically **modify HTML content and CSS styles** using JavaScript
- Handle different types of **HTML DOM events**, including:
  - Mouse events
  - Keyboard events
  - Form events
  - Object events
- Implement **image replacement** and **preload images** for better performance
- Use **setTimeout()** to execute delayed actions
- Use **addEventListener()** for clean and flexible event handling
- Build a **dynamic data table** that displays user input and supports **row deletion**

2. Starter Files

Base HTML File (index.html)

Use **the given HTML exactly as below**, with **only two small additions**:

1. An empty table for registered travelers
2. A script reference to an external JS file

Add the following below the form:

```html
<hr>
<h2>Registered Travelers</h2>

<table border="1" cellpadding="8" cellspacing="0" id="travelerTable">
    <tr>
        <th>Name</th>
        <th>Age</th>
        <th>Email</th>
        <th>Action</th>
    </tr>
</table>

<script src="script.js"></script>
```

Why?

- The table will be populated dynamically using DOM methods
- JavaScript is separated from HTML (best practice)

3. Creating the JavaScript File

Create a new file named:

`script.js`

All DOM logic must go **only inside this file**.

4. Understanding the DOM Tree (Conceptual Step)

Before writing any JavaScript, it is important to understand how the browser represents the HTML page internally.

When a web page loads, the browser converts the HTML into a **Document Object Model (DOM)**.

The DOM represents the page as a **tree of objects**, where each HTML element becomes a node.

Key DOM Objects in This Page

- **document**
  Represents the entire web page.
  All DOM operations start from this object.
- **document.body**
  Represents everything inside the `<body>` tag, including headings, images, tables, and forms.
- **document.images**
  Represents a collection of all `<img>` elements in the page.
  Images are accessed using index values (e.g., document.images[0]).
- **document.forms**
  Represents a collection of all `<form>` elements in the page.
  Forms and their input fields are accessed through this object.

5. DOM Methods: Finding HTML Elements

In this part, you will learn how to **locate HTML elements using DOM methods**.

Before JavaScript can modify an element, it must first **find that element in the DOM tree**.

Access Elements by Tag and Document Collections

Open the script.js file and add the following code:

```
// DOM Methods
let title = document.getElementsByTagName("h1")[0];
let image = document.images[0];
let form = document.forms[0];
```

Explanation

- **document.getElementsByTagName("h1")**
  Returns a collection of all `<h1>` elements in the page.
  Since there is only one `<h1>`, it is accessed using index [0].
- **document.images**
  Returns a collection of all `<img>` elements in the document.
  The travel image is accessed using document.images[0].
- **document.forms**
  Returns a collection of all `<form>` elements in the document.
  The traveler information form is accessed using document.forms[0].

Key Points to Remember

- These methods return **collections**, not single elements
- Indexing ([0]) is required to access individual elements
- This approach reflects how elements are organized in the DOM tree

6. Document Object Properties

In this part, you will learn how to **access and modify properties of the document object**.

These properties provide information about the web page and allow basic page-level updates.

Read Document Properties on Page Load

Add the following code to script.js:

```
window.onload = function () {
    console.log("Page URL:", document.URL);
    console.log("Last Modified:", document.lastModified);
    document.title = "Travel Adventures - DOM Lab";
};
```

Explanation

- **window.onload**
  Ensures the code runs only after the entire page has loaded.
- **document.URL**
  Returns the URL of the current web page.
- **document.lastModified**
  Returns the date and time the document was last modified.
- **document.title**
  Changes the title shown in the browser tab.

```
Page URL:                                                                    script.js:14
file:///C:/Users/Manuka/OneDrive%20-%20Sri%20Lanka%20Institute%20of%20Information%20Technology/Desktop/html-dom-lab/index.html
Last Modified: 12/15/2025 14:15:20                                           script.js:15
Window resized                                                               script.js:115
```

7. Changing HTML Styles Using the DOM Style Object

In this part, you will learn how to **change the appearance of HTML elements dynamically** using the DOM **style object**.

Change Text Style Dynamically

Add the following code to script.js:

```
title.style.color = "darkblue";
title.style.fontSize = "36px";
title.style.fontFamily = "Arial";
```

Explanation

- **element.style**
  Allows JavaScript to directly modify CSS properties of an HTML element.
- **CSS property names**
  Are written in **camelCase** in JavaScript:
  - font-size → fontSize
  - font-family → fontFamily
- These style changes are applied **immediately** without reloading the page.

8. Background Properties Using the DOM

In this part, you will learn how to **change background properties dynamically** and use a **time delay** in JavaScript.

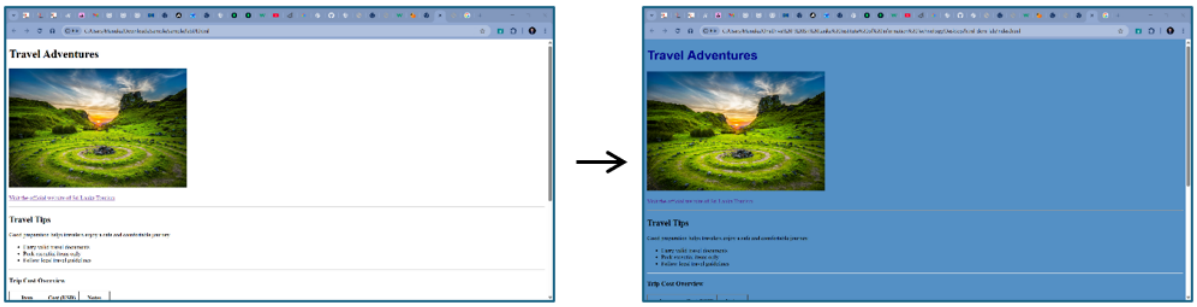
Change Page Background After 2 Seconds

Add the following code to script.js:

```
setTimeout(function () {
    document.body.style.backgroundColor = "#5490c5ff";
}, 2000);
```

Explanation

- **setTimeout()**
  Executes a function after a specified delay (in milliseconds).
- **2000 milliseconds**
  Equals a delay of **2 seconds**.
- **document.body.style.backgroundColor**
  Changes the background color of the entire page dynamically.



Before *After 2 seconds*

9. HTML DOM Events

In this part, you will learn how JavaScript reacts to **user actions** using **HTML DOM events**.

Events allow your page to respond dynamically when users interact with elements.

9.1 Mouse Events

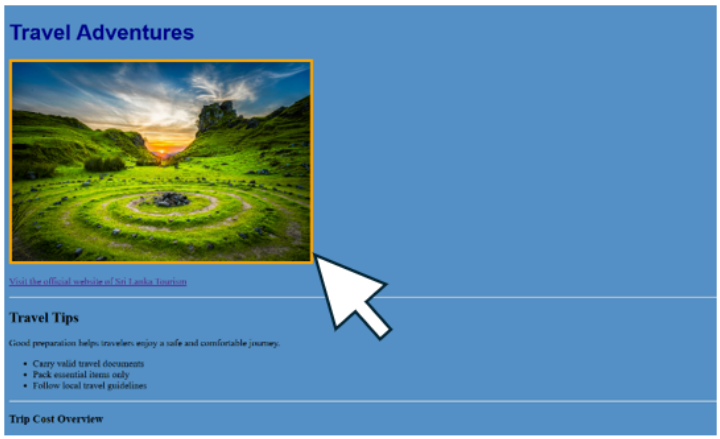
Mouse events are triggered when the user interacts with the mouse.

Add the following code to script.js:

```
image.onmouseover = function () {
    image.style.border = "5px solid orange";
};

image.onmouseout = function () {
    image.style.border = "none";
};
```

Explanation

- **onmouseover** triggers when the mouse pointer moves over the image
- **onmouseout** triggers when the mouse pointer leaves the image
- The image border changes dynamically based on mouse movement



9.2 Keyboard Events

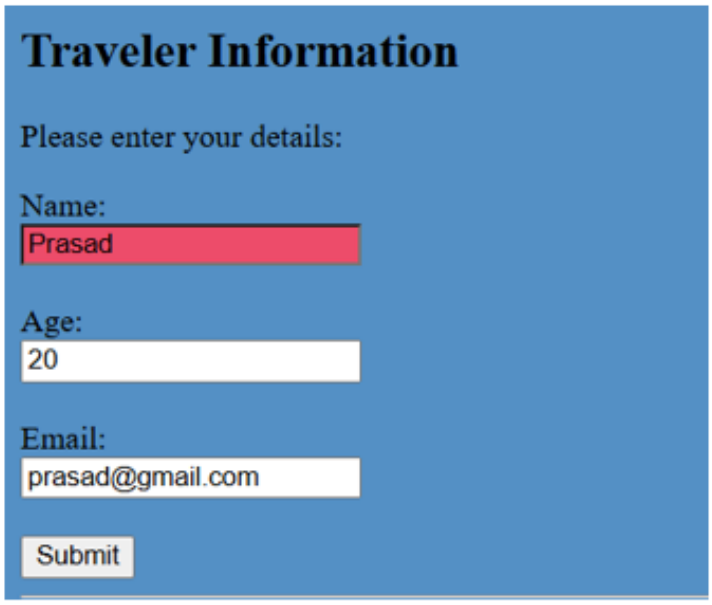
Keyboard events occur when the user types using the keyboard.

Add the following code:

```
document.getElementById("name").onkeyup = function () {
    this.style.backgroundColor = "#ed4c6aff";
};
```

Explanation

- **onkeyup** triggers when a key is released
- This event occurs while typing inside the **Name** input field
- The input background color changes as visual feedback



9.3 Form Events

Form events are triggered when a user interacts with a form, especially during submission.

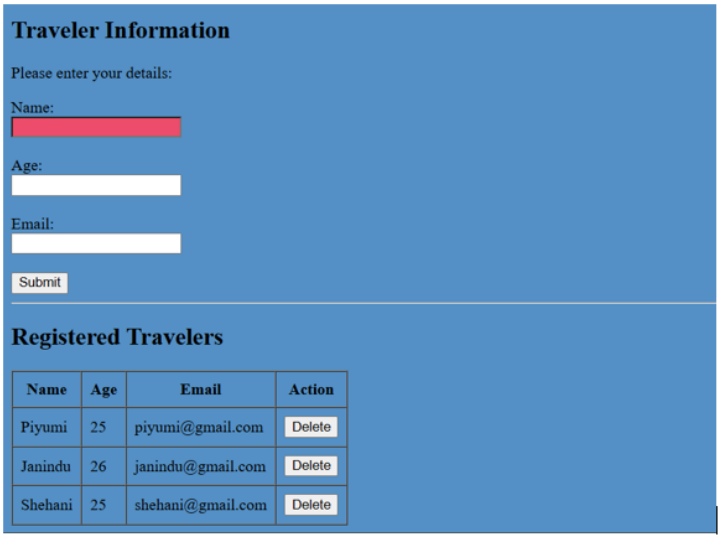
Step 9.3.1: Handle Form Submission

Add the following code:

```
form.onsubmit = function (event) {
    event.preventDefault();
    addTraveler();
};
```

Explanation

- **onsubmit** triggers when the form is submitted
- **event.preventDefault()** stops the page from refreshing
- The addTraveler() function will handle form data using the DOM



10. The addEventListener() Method

In this part, you will learn a **better and more flexible way** to handle events using the addEventListener() method.

This method allows JavaScript to listen for user actions **without modifying HTML event attributes**.

Update the HTML (Add an id to the Table)

To attach an event listener, the element must be uniquely identifiable in the DOM.

Locate the **Trip Cost Overview table** in index.html and **add an id attribute** as shown below:

```html
<table border="1" cellpadding="8" cellspacing="0" id="costTable">
```

Add the Event Listener Using JavaScript

Now open script.js and add the following code:

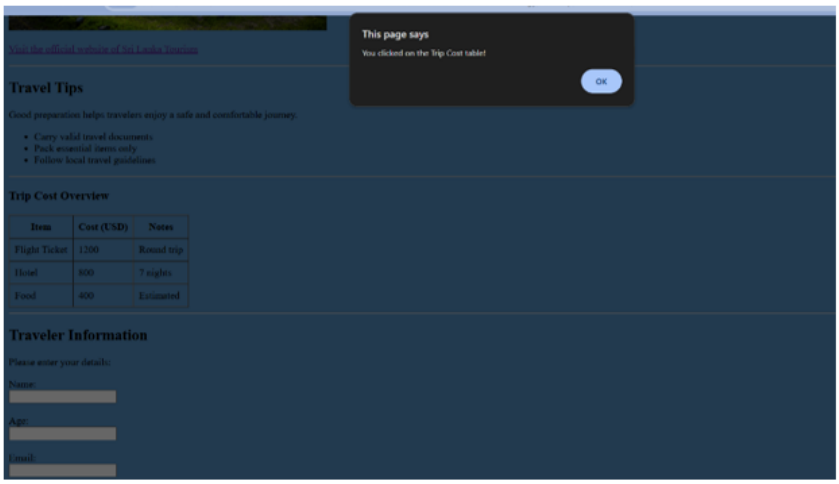
```javascript
let costTable = document.getElementById("costTable");

costTable.addEventListener("click", function () {
    alert("You clicked on the Trip Cost table!");
});
```

Explanation

- **id="costTable"** uniquely identifies the table in the DOM
- **document.getElementById()** is used to access the table
- **addEventListener()** attaches a click event handler to the table
- The function executes when the user clicks anywhere inside the table

Why Use addEventListener()?

- It does **not overwrite** existing event handlers
- Multiple listeners can be attached to the same element
- Keeps **HTML and JavaScript separate**
- Improves readability and maintainability



11. Image Replacement and Preloading

In this part, you will learn how to **preload images into the browser cache** and **replace an image dynamically** using the DOM.

11.1: Preload Images

Add the following code to script.js:

```
let img1 = new Image();
let img2 = new Image();

img1.src = "./images/travel2.jpg";
img2.src = "./images/travel1.jpg";
```

Explanation

- **new Image()** creates an image object in memory
- Setting the src property starts loading the image immediately
- Images are loaded into the **browser cache** before being displayed

This improves performance when images are replaced.

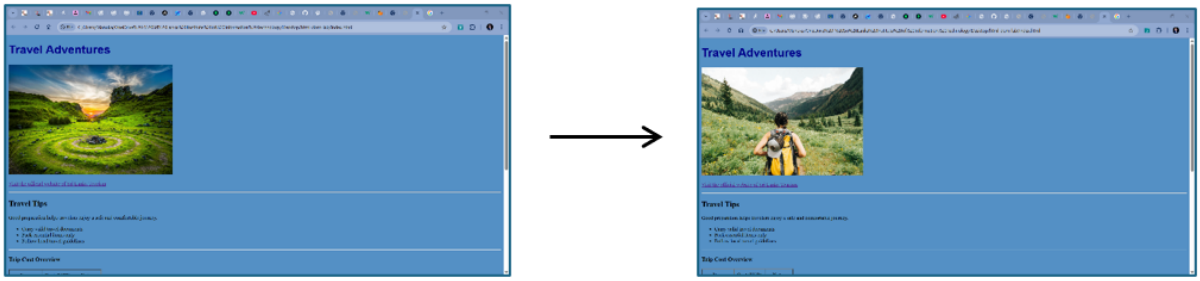
11.2: Replace Image on Double Click

Add the following code:

```
image.ondblclick = function () {
    image.src = img2.src;
};
```

Explanation

- **ondblclick** triggers when the user double-clicks the image
- The src attribute of the existing image is replaced
- The image change happens instantly because it is already preloaded



Before double click *After double click*

12. Display Submitted Form Data in a Table

In this part, you will use the DOM to **capture form data** and **display it dynamically in a table** on the same page.

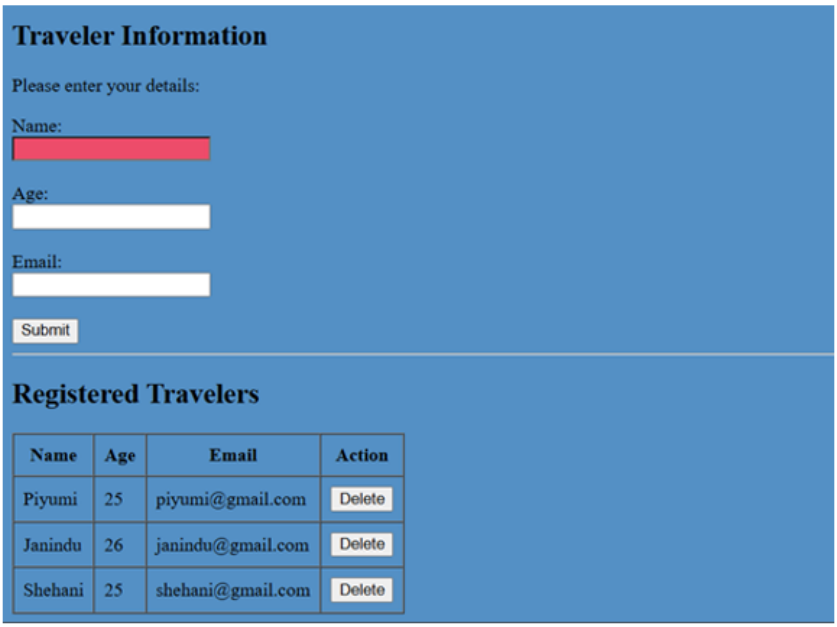
Add a Traveler to the Table

Add the following function to script.js:

```
function addTraveler() {
    let name = document.getElementById("name").value;
    let age = document.getElementById("age").value;
    let email = document.getElementById("email").value;

    let table = document.getElementById("travelerTable");
    let row = table.insertRow();

    row.insertCell(0).innerText = name;
    row.insertCell(1).innerText = age;
    row.insertCell(2).innerText = email;

    let deleteCell = row.insertCell(3);
    let btn = document.createElement("button");
    btn.innerText = "Delete";
    btn.onclick = function () {
        table.deleteRow(row.rowIndex);
    };

    deleteCell.appendChild(btn);

    form.reset();
}
```

Explanation

- Form input values are read using **getElementById().value**
- A new row is added to the table using **insertRow()**
- Table cells are added using **insertCell()**
- A **Delete** button is created dynamically using **createElement()**
- Clicking the Delete button removes the corresponding table row
- The form is cleared after submission using **form.reset()**



13. Object Events

In this part, you will learn how to handle **object-level events**, which are events triggered by browser objects such as the window.

Handle Window Resize Event

Add the following code to script.js:

```
window.onresize = function () {
    console.log("Window resized");
```

Explanation

- **window** is a browser object
- **onresize** triggers whenever the browser window is resized
- The message is logged to the console each time the event occurs

In Class Test 04 – HTML DOM with jQuery **30 minutes**

You are given a **base HTML file**.

You MUST:

- Use **jQuery only** for DOM manipulation
- **NOT modify the HTML file**, except where explicitly mentioned
- Write **all DOM logic in a JavaScript file**
- Add your **Student ID and Name** at the top of the JS file
- Rename the JS file using your **Student ID**
- Upload **only the JS file**

STEP 1 – Create Your JavaScript File (MANDATORY)

1. Create a JavaScript file
2. Rename it using your Student ID
3. **ITXXXXXXXX.js**
4. At the **top of the file**, add:

```
/*
Student ID: ITXXXXXXXX
Name: Your Full Name
*/
```

STEP 2 – Base HTML File (GIVEN – DO NOT CHANGE)

Save as index.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Library Seat Booking</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>

<h1 id="pageTitle">University Library Seat Booking</h1>

<img id="libraryImage" src="./images/library1.jpg" width="400">

<p id="infoText">
    Reserve your study seat in advance.
</p>

<hr>

<h2>Book a Seat</h2>

<form id="bookingForm">
    <label>Student Name:</label><br>
    <input type="text" id="studentName"><br><br>

    <label>Student Email:</label><br>
    <input type="email" id="studentEmail"><br><br>
```

```html
    <input type="submit" value="Book Seat">
</form>

<hr>

<h2>Seat Categories</h2>

<table border="1" cellpadding="8" cellspacing="0" id="seatTable">
    <tr>
        <th>Seat Type</th>
        <th>Floor</th>
    </tr>
    <tr>
        <td>Silent Study</td>
        <td>1st Floor</td>
    </tr>
    <tr>
        <td>Group Study</td>
        <td>2nd Floor</td>
    </tr>
</table>

<hr>

<h2>Booked Seats</h2>

<table border="1" cellpadding="8" cellspacing="0" id="bookingTable">
    <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Action</th>
    </tr>
</table>

</body>
</html>
```

TASK 1 – Page Load & Document Properties

Using **jQuery**:

1. Run your code only **after the page is fully loaded**
2. Change the browser tab title to **"Library Seat Booking – DOM Test"**
3. Print the **current page URL** to the console

TASK 2 – Style & Background Manipulation

Using **jQuery**:

1. Change the page title color to **dark green** *(#147d1b)*
2. Change its font size to **34px**
3. After **2 seconds**, change the **page background color** to **light gray** *(#D3D3D3)*

TASK 3 – Mouse Events & addEventListener Concept

Using **jQuery events**:

1. When the mouse is moved **over the library1 image**, add a **5px solid blue border** *(#0000ff)*
2. When the mouse moves **out**, remove the border
3. When the user **clicks on the Seat Categories table**, display an alert message

TASK 4 – Keyboard & Object Events

Using **jQuery**:

1. When the user **types in the Student Name field**, change its background color to **light yellow** *(#FFFF00)*
2. When the **browser window is resized**, print "**Window resized**" to the console

TASK 5 – Image Preloading & Image Replacement

Using **jQuery and JavaScript**:

1. Preload a second image (library2.jpg)

2. When the user **double-clicks the library1 image**, replace it with the preloaded image
3. Double-clicking again should toggle back to the original image

TASK 6 – Form Handling & Dynamic Table

Using **jQuery**:

1. Prevent the form from refreshing the page
2. Read **Student Name** and **Student Email**
3. Add the data to the **Booked Seats table**
4. Add a **Delete button** for each row
5. Clicking Delete must remove the corresponding row